# ■ Raitha-Bharosa Hub

Smart Sowing Assistant for the Modern Indian Farmer

*Android · Jetpack Compose · MVVM · GenAI · Kannada / English*

| Android App | Agriculture | GenAI Powered | Open Source |
|---|---|---|---|

Raitha-Bharosa Hub is a decision-support system that bridges atmospheric data and the farmer's field. It translates complex soil and weather inputs into clear, localized, actionable farming steps — available in Kannada and English — so every farmer, regardless of digital literacy, can make the right call at the right time.

## 01 The Problem

Farmers across India still rely on generational intuition for decisions about sowing and fertilization. That intuition, built over decades, was reliable when weather patterns were predictable. Today it is not.

| | |
|---|---|
| **Shifting Weather** | Erratic rainfall and temperature swings make traditional seasonal cues misleading. |
| **Soil Depletion** | Repeated cropping without data-driven fertilization depletes N-P-K levels silently. |
| **The Missed Window** | There exists a critical 48-hour window where soil moisture and temperature align perfectly for germination. Without a tool, farmers miss it — and lose yields. |
| **No Local Tool** | Existing agri-apps are designed for urban agronomists, not for a farmer standing in a field in rural Karnataka. |

## 02 The Vision

Raitha-Bharosa Hub is not a weather app. It is a Smart Sowing Assistant. A farmer registers their specific plot, selects their crop — Sugarcane, Ragi, or Paddy — and the app continuously monitors what we call the Crop Health Velocity.

By comparing real-time (or simulated) weather and soil inputs against established agricultural benchmarks, the app distills everything into one number: the Sowing Index. Green means go. Red means wait. The farmer never has to interpret raw data.

## 03 How a Farmer Uses the App

**1** **Onboarding**
The farmer opens the app and is greeted in Kannada or English. They enter their name and select their primary crop. That is all — no long forms, no technical jargon.

**2** **The Dashboard**
The main screen shows the current Sowing Index as a large percentage dial. Color coding (Green / Amber / Red) gives an instant verdict at a glance.

**3** **Input Center**
Farmers can manually log soil test results such as N-P-K levels or record field moisture observations from a basic soil probe.

**4** **Krishi Calendar**
A 7-day action plan shows exactly what to do each day. If a storm is predicted in three days, the app automatically warns the farmer to finish fertilization 24 hours earlier.

**5** **Season History**
A local log stores what the farmer did in previous seasons alongside recorded yields — building a personal data set that improves advice over time.

## 04 Technical Implementation

**Architecture** **MVVM (Model-View-ViewModel)**

The app is built on a strict MVVM pattern. The ViewModel holds and processes all business logic — the Sowing Index calculation, soil evaluation rules — while Jetpack Compose observes state and redraws the UI automatically whenever data changes.

**UI Framework** **Jetpack Compose**

The entire interface is built in Compose with large tap targets, high-contrast icons, and a two-color decision system. Green means conditions are ideal. Red means the farmer should wait. No reading required.

**Data Persistence** **Room Database**

All farmer profiles, crop selections, historical logs, and simulated soil parameters are stored locally using Room. The app works offline — critical in rural areas with patchy connectivity.

**Simulation Engine** **DataGenerator Class**

A dedicated DataGenerator class produces realistic randomized sensor readings. For example, it generates soil moisture between 10 and 40 percent. The app then evaluates: if moisture exceeds 30 percent, it displays Soil too wet to sow. This keeps the logic testable without real hardware.

**Weather Integration** **Retrofit and OpenWeatherMap**

Retrofit handles all HTTP communication. Live weather data is fetched from OpenWeatherMap. Where API credits are limited, a local JSON file provides mock weather data so the decision engine always has inputs to work with.

## 05 Impact Goals

| Goal | Description | Target |
|---|---|---|
| **Precision Farming** | Better sowing timing means seeds go into the ground only when conditions are optimal. | 15 to 20% less seed wastage |
| **Digital Literacy** | Engineering solutions delivered at the grassroots level, bridging the rural-urban tech gap. | Accessible to first-time smartphone users |
| **Sustainability** | Data-driven fertilizer recommendations prevent over-application and soil degradation. | Targeted NPK use only when needed |

## 06 Success Criteria

| | |
|---|---|
| **Performance** | The main dashboard loads within 2 seconds on a mid-range Android device. |
| **Responsiveness** | The Sowing Index updates dynamically every time simulated sensor data changes. |
| **Accessibility** | The full interface is available in Kannada with no functionality gap versus English. |
| **Code Quality** | The codebase follows MVVM architecture with clear separation between data, logic, and UI layers. |